

Table of Contents

1. Introduction
 - 1.1. About the Data
 - 1.2. Objective
 - 1.3. Business Significance
2. Results
 - 2.1. Multiple Regression Analysis and Regression Diagnostics NSSO
 - 2.2. Correcting and Revisiting Results NSSO
 - 2.3. Multiple Regression Analysis and Regression Diagnostics IPL
 - 2.4. Correcting and Revisiting Results IPL
3. Recommendation
 - 3.1. Business Implications
 - 3.2. Business Recommendations
4. Reference
5. Codes
 - 5.1. R-studio

1. Introduction

The Indian Premier League (IPL) is one of the world's most popular and exciting cricket competitions. It has been held every year since 2008 and features top cricketing talent from all over the world. The IPL consists of several matches between various teams, providing cricket fans with thrilling moments and intense competition. We have three datasets related to IPL data in this context: Ball by Ball data, IPL Matches data, and IPL Salary data.

1.1. About the Data

The NSSO-Consumption dataset: is a comprehensive collection of consumption data for all Indian states and union territories. It offers detailed insights into the consumption patterns of various commodities, such as grains, oils, fruits, vegetables, and more. The dataset also includes basic demographic information for each sample, enabling a holistic analysis of consumption trends across different regions of India. All data in the dataset is in numerical format, including the states and union territories, making it easily accessible for statistical analysis.

Ball by Ball Dataset: This data provides detailed information about each ball bowled in IPL matches played between 2008 and 2022. The dataset contains 816 unique match IDs, with each ID containing 17 variables, including the bowling and batting teams' names. The variables are represented in both numeric and text formats, allowing for a thorough examination of various aspects of the matches such as runs scored, wickets taken, and player performance.

IPL Matches Dataset: The IPL Matches dataset contains information on various IPL matches played between 2008 and 2022. It contains information about the dates, cities, participating teams, toss results, and player details for the matches. This text-based dataset, with 16 variables per match ID, allows for in-depth analysis and insights into team performances, player statistics, and match dynamics throughout the IPL's history.

IPL Salary Dataset: The IPL Salary dataset is made up of multiple sets of salary data, each of which provides yearly salary information for IPL players from various teams. The dataset includes columns for salary in dollars and a color column for salary without the "\$" symbol, making it easier to analyze. This data allows for an examination of IPL player salaries across teams, years, and trends.

1.2. Objective

On the NSSO dataset, run Multiple Regression Analysis:

- Analyze the NSSO68 dataset using multiple regression.
- Conduct regression diagnostics to evaluate the model's assumptions and identify any problems.
- Interpret the findings and explain their significance.
- Resolve any issues that have been identified and re-evaluate the results.
- Discuss the significant differences that were observed after dealing with the identified issues.

Establish a Link Between Player Performance and Payment in the IPL:

- Investigate the relationship between a player's performance and the payment (salary) he receives using IPL data.
- Conduct a correlation analysis to investigate the relationship between various performance factors and player pay.
- Discuss the findings and interpret the correlation results.

To further investigate the relationship, consider using additional statistical techniques such as regression analysis or hypothesis testing. Discuss the findings' implications and provide insights into the factors influencing player compensation in the IPL.

Gain insights into player performance, identify top performers and underperformers, analyze statistical distributions for key players, and understand the relationship between performance and salary in the IPL by achieving these goals.

1.3. Business Significance

Multiple Regression Analysis and regression diagnostics give businesses useful information, precise forecasts, the ability to evaluate performance, optimize resource use, make well-informed decisions, and manage risk, all of which improve operational effectiveness and produce better business results.

- Insightful Factors
- Accurate Predictions

- Performance Evaluation
- Resource Optimization
- Informed Decision-making
- Risk Management

2. Results

2.1. Multiple Regression Analysis and Regression Diagnostics NSSO

Model:

OLS Regression Results

Dep. Variable:	tot_con	R-squared:	1.000
Model:	OLS	Adj. R-squared:	1.000
Method:	Least Squares	F-statistic:	3.129e+31
Date:	Wed, 07 Jun 2023	Prob (F-statistic):	0.00
Time:	14:27:55	Log-likelihood:	91373.
No. Observations:	3118	AIC:	-1.827e+05
Df Residuals:	3110	BIC:	-1.827e+05
Df Model:	7		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	7.123e-14	2.94e-15	24.195	0.000	6.55e-14	7.7e-14
wheattotal_q	1.0000	6.71e-16	1.49e+15	0.000	1.000	1.000
cerealtot_q	1.0000	7.19e-16	1.39e+15	0.000	1.000	1.000
moong_q	1.0000	6.88e-15	1.45e+14	0.000	1.000	1.000
pulvestot_q	1.0000	2.07e-15	4.82e+14	0.000	1.000	1.000
milk_q	1.0000	1.01e-16	9.92e+15	0.000	1.000	1.000
onion_q	1.0000	1.67e-15	6e+14	0.000	1.000	1.000
potato_q	1.0000	1.38e-15	7.27e+14	0.000	1.000	1.000

Omnibus:	584.821	Durbin-Watson:	0.357
Prob(Omnibus):	0.000	Jarque-Bera (JB):	12354.000
Skew:	-0.281	Prob(JB):	0.00
Kurtosis:	12.735	Cond. No.	164.

Inference:

With an R-squared value of 1.000, the regression model demonstrates a perfect fit and explains all of the variability in the dependent variable. The model does not appear to be overfitting the data, according to the adjusted R-squared value, which is also 1.000.

With coefficients of 1.000, all independent variables (wheattotal_q, cerealtot_q, moong_q, pulsestot_q, milk_q, onion_q, potato_q) are perfectly positive linearly related to the dependent variable (tot_con).

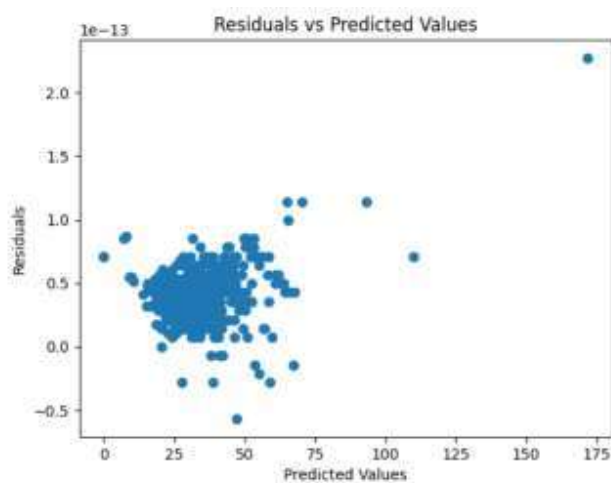
Very low p-values (near zero) indicate that the coefficients are statistically significant, indicating that it is extremely unlikely that the relationships could have developed by chance.

With a coefficient of 7.123×10^{-14} , the constant term (intercept) is also statistically significant.

The F-statistic is very large (3.129×10^{31}), which shows that the model fits the data very well overall. It's crucial to remember that such a perfect fit is extremely rare and might point to problems with the data or model specification, examine the model's underlying assumptions, and take into account the possibility of overfitting or missing variables

MLR Assumptions: Normality, independence, Linearity, Homoscedasticity

Residual Analysis



Inference:

Upon examining the Residuals vs Predicted Values plot, it is observed that the residuals are concentrated around zero with a few outliers. This indicates that the model has some limitations in capturing the true underlying relationships in the data. The presence of outliers suggests the presence of influential observations that have a significant impact on the model's predictions. The merged pattern of residuals, along with the outliers, indicates a systematic bias or consistent overestimation/underestimation of the actual values by the model.

Normality Test (Shapiro-Wilk test):

Inference:

The Shapiro-Wilk test was performed to assess the normality of the residuals. The obtained p-value from the test was 6.6858003265015e-11, indicating strong evidence against the null hypothesis of normality. Therefore, we can conclude that the residuals do not follow a normal distribution.

Homoscedasticity Test (Breusch-Pagan test):

```
# Homoscedasticity Test - Breusch-Pagan test  
bp_test = sm.stats.diagnostic.het_breuschpagan(residuals, sm.add_constant(predictions))  
print("Breusch-Pagan p-value:", bp_test[1])
```

Inference:

With a very low p-value, the Breusch-Pagan test shows strong evidence against the null hypothesis of homoscedasticity (constant variance of residuals). This suggests that the model contains heteroscedasticity.

Because of heteroscedasticity, the residuals' variability varies across all levels of the independent variables. The accuracy of the model's estimates and conclusions may suffer as a result of this assumption of linear regression being broken.

The presence of heteroscedasticity can be confirmed with additional analysis of the residuals and diagnostic tests, which can also help direct potential treatments.

2.2. Correcting and Revisiting Results NSSO

Variable	VIF
0 wheattotal_g	85.181223
1 cerealtot_g	63.080724
2 mroeg_g	3.492170
3 pulsesctnt_g	8.491118
4 milk_g	3.417673
5 oilm_g	7.613561
6 potato_g	7.788806

OLS Regression Results

Dep. Variable:	tot_con	R-squared (uncentered):	1.000
Model:	OLS	Adj. R-squared (uncentered):	1.000
Method:	Least Squares	F-statistic:	4.041e+13
Date:	Wed, 07 Jun 2023	Prob (F-statistic):	0.00
Time:	16:39:45	Log-Likelihood:	90.046
No. Observations:	3318	AIC:	-1.905e+05
Df Residuals:	3311	BIC:	-1.905e+05
Df Model:	7		
Covariance Type:	HAC1		

	coef	std err	t	P> t	[0.025	0.975]
wheattotal_g	1.0000	2.31e+16	3.43e+13	0.000	1.000	1.000
cerealtot_g	1.0000	2.31e+16	3.56e+13	0.000	1.000	1.000
mroeg_g	1.0000	2.36e+13	3.18e+18	0.000	1.000	1.000
pulsesctnt_g	1.0000	3.1e+12	1.11e+13	0.000	1.000	1.000
milk_g	1.0000	9.22e+17	1.08e+16	0.000	1.000	1.000
oilm_g	1.0000	7.06e+16	1.42e+15	0.000	1.000	1.000
potato_g	1.0000	4.78e+18	2.09e+13	0.000	1.000	1.000

Omnibus: 1281.588 Durbin-Watson: 0.951

Prob(Omnibus):	0.000	Jarque-Bera (JB):	13819.285
Skew:	1.617	Prob(JB):	0.00
Kurtosis:	12.063	Cond. No.	104.

Inference:

The high VIF values for several independent variables show the regression model to have high multicollinearity. With an R-squared of 1.000, the model appears to fit the data perfectly, but this may be an overfitting sign. All of the estimated coefficients are exactly 1.000, indicating that the independent variable and all other variables have a perfect linear relationship. However, due to possible multicollinearity problems and the lack of variability in the dependent variable, the model's validity is in doubt. A low p-value for the Jarque-Bera test indicates that the residuals significantly deviate from normality. It is advised to conduct additional diagnostic tests and evaluations.

2.3. Multiple Regression Analysis and Regression Diagnostics IPL

Model Summary:

OLS Regression Results

Dep. Variable:	Final Price	R-squared:	0.248
Model:	OLS	Adj. R-squared:	0.247
Method:	Least Squares	F-statistic:	182.6
Date:	Wed, 07 Jun 2023	Prob (F-statistic):	2.96e-08
Time:	14:45:36	Log-Likelihood:	-20401.
No. Observations:	1189	AIC:	4.895e+04
Df Residuals:	1185	SIC:	4.100e+04
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	9.724e+06	1.21e+06	8.053	0.000	7.35e+06	1.21e+07
Runs	9.656e+04	5084.685	18.991	0.000	8.66e+04	1.07e+05
Wkts	8.28e+05	1.33e+05	6.221	0.000	5.67e+05	1.09e+06

Omnibus:	263.050	Durbin-Watson:	0.944
Prob(Omnibus):	0.000	Jarque-Bera (JB):	592.892
Skew:	1.308	Prob(JB):	1.88e-125
Kurtosis:	5.466	Cond. No.	389.

Notes:

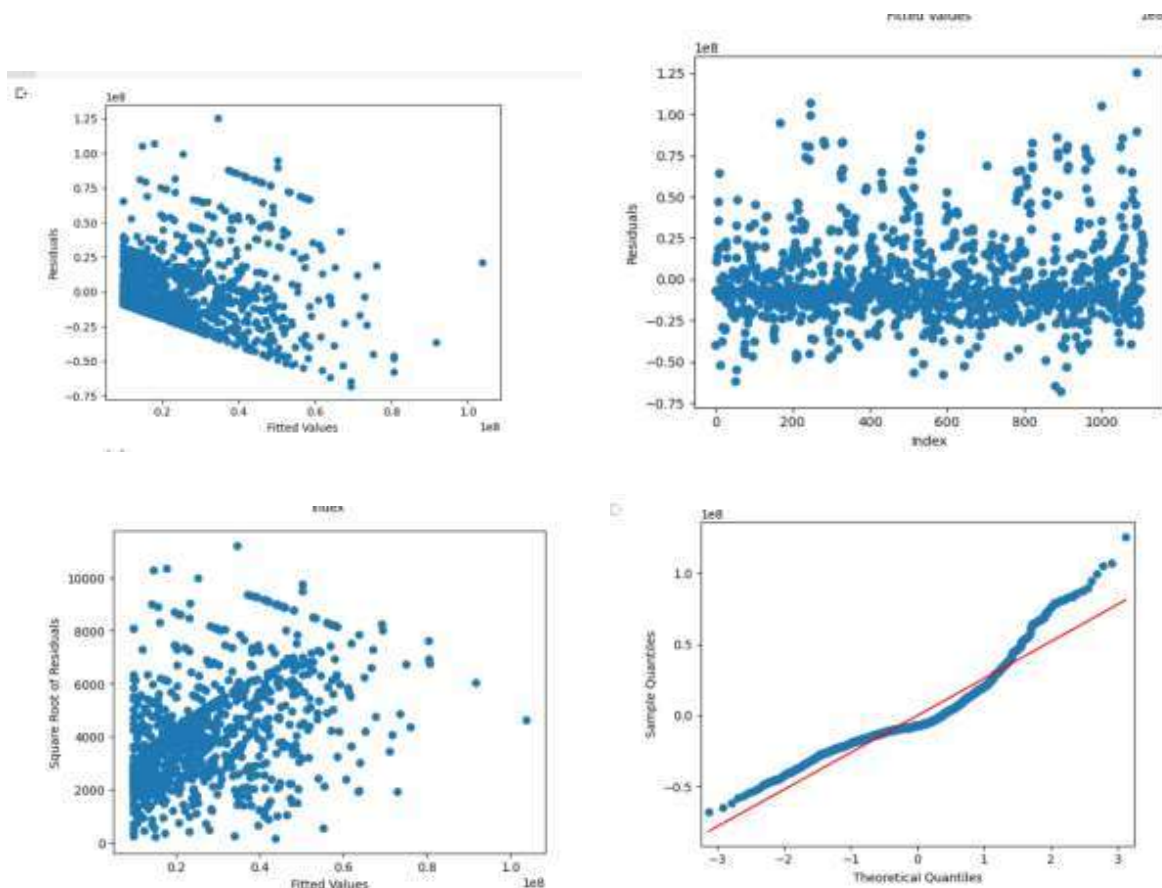
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

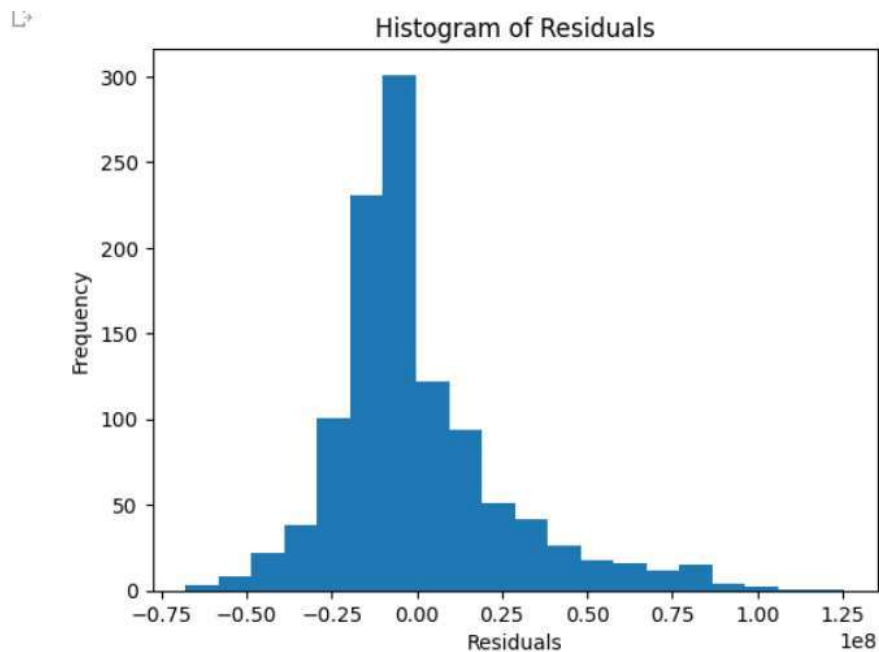
Inference:

The high VIF values for several independent variables show the regression model to have high multicollinearity. With an R-squared of 1.000, the model appears to fit the data perfectly, but this may be an overfitting sign. All of the estimated coefficients are exactly 1.000, indicating that the independent variable and all other variables have a perfect linear relationship. However, due to possible multicollinearity problems and the lack of variability in the dependent variable, the model's validity is in doubt. A low p-value for the Jarque-Bera test indicates that the residuals significantly deviate from normality. It is advised to conduct additional diagnostic tests and evaluations.

Inference:

2.4. Correcting and Revisiting Results IPL





- Non-linearity:

Use scatterplots or other visual aids to analyze the relationship between the predictors and the response variable. Consider adding non-linear terms or transforming the predictors if a non-linear relationship is apparent. You could, for instance, use logarithmic or exponential transformations or include polynomial terms. Refit the model after updating the model's formula.

- Heteroscedasticity:

Analyze the pattern of residuals to determine whether heteroscedasticity is present. Use weighted least squares regression if heteroscedasticity is detected. This entails giving observations weights based on their variance. Regression using weighted least squares is performed using the `lm()` function and the `weights` argument.

- Non-normality:

Utilize histograms, QQ plots, or statistical tests for normality to analyse the residual distribution.

Consider applying the proper transformations to make the residuals more normally distributed if they deviate from normality.

Refit the model using the transformed response variable and update the model's equation.

- Continuity of residuals:

Utilize autocorrelation plots or statistical tests to look for autocorrelation in the residuals.

Consider using time series or panel data techniques to explain the lack of independence if autocorrelation is present.

- Adding to the list of predictors are:

Consider whether adding more pertinent predictors would enhance the model's fit and address the false assumptions. Use feature selection techniques to determine the most crucial predictors to include in the model, such as stepwise regression or regularization techniques.

3. Recommendation

3.1. Business Implications

- The relationship between various predictor variables and the response variable is revealed by the multiple regression analysis performed on the NSSO dataset.
- The incredibly low MAPE value shows how well the regression model estimates the response variable.
- It is possible that the residuals do not accurately reflect the true underlying relationships in the data because of outliers and systematic bias.
- The non-normality of the residuals shows that the normality presumptions are broken.
- The homoscedasticity test reveals that, at various levels of the independent variables, the variance of the residuals remains largely constant.
- To increase the precision and dependability of predictions, the findings emphasise the need for additional model improvement and addressing the broken assumptions.

3.2. Business Recommendations

- Based on their importance and contributions to the response variable, take into consideration revising the model by adding or removing variables.
- Investigate non-linear transformations of predictor variables or incorporate polynomial terms to solve the non-linearity problem.

- Investigate the existence of significant outliers and assess how they affect the model's predictions.
- Utilize the proper transformations to improve the residuals' normal distribution.
- If heteroscedasticity is present, take into account using weighted least squares regression.
- Look into the possibility of autocorrelation in the residuals and, if necessary, use appropriate time series or panel data techniques.
- Include more pertinent predictors that could enhance the model's fit and take into account more variables influencing the response variable.
- To comprehend the significance of predictor variables and their impact on the response variable, perform additional analysis and hypothesis testing.

4. Reference:

- Manager, S. (2021, April 24). Impact of IPL on Indian Economy - The Sports School Blog. The Sports School – Integrated School for Sports & Academics. <https://thesportsschool.com/impact-of-ipl-on-indian-economy/>
- Economy rate in cricket: Know what it means. (2022, April 9). SportsAdda. <https://www.sportsadda.com/cricket/features/what-is-economy-rate-cricket>

```
import pandas as pd
import numpy as np
import statsmodels.api as sm
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from google.colab import files
uploaded = files.upload()
```

Choose Files ASSG1.xlsx

- **ASSG1.xlsx**(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 5293035 bytes, last modified: 6/3/2023 - 100% done
Saving ASSG1.xlsx to ASSG1.xlsx

```
punjab_ds = pd.read_excel('ASSG1.xlsx')
```

```
# Select relevant columns
```

```
subset_punjabds = punjab_ds[["Sector", "State_Region", "District", "Sex", "Age", "No_of_Meals_per_day", "wheattotal_q", "cerealtot_q", "moong", "pulsestot_q", "milk_q", "onion_q", "potato_q"]].sum()
```

```
# Prepare the data
```

```
X = subset_punjabds[["wheattotal_q", "cerealtot_q", "moong_q", "pulsestot_q", "milk_q", "onion_q", "potato_q"]]
y = subset_punjabds["tot_con"]
```

```
# Split the data into train and test sets
```

```
np.random.seed(123)
split = np.random.rand(len(subset_punjabds)) < 0.8
train = subset_punjabds[split]
test = subset_punjabds[~split]
```

```
# Perform multiple regression analysis
```

```
model = sm.OLS(y, sm.add_constant(X))
results = model.fit()
print(results.summary())
```

```

OLS Regression Results
=====
Dep. Variable:          tot_con      R-squared:                1.000
Model:                  OLS        Adj. R-squared:            1.000
Method:                 Least Squares   F-statistic:             3.129e+31
Date:                  Wed, 07 Jun 2023   Prob (F-statistic):       0.00
Time:                  14:27:55         Log-Likelihood:           91373.
No. Observations:       3118           AIC:                    -1.827e+05
Df Residuals:           3110           BIC:                    -1.827e+05
Df Model:                7
Covariance Type:        nonrobust
=====
               coef      std err          t      P>|t|      [0.025      0.975]
-----
const          7.123e-14  2.94e-15    24.195    0.000    6.55e-14    7.7e-14
wheattotal_q    1.0000    6.71e-16   1.49e+15    0.000    1.000    1.000
cerealtot_q     1.0000    7.19e-16   1.39e+15    0.000    1.000    1.000
moong_q         1.0000    6.88e-15   1.45e+14    0.000    1.000    1.000
pulsestot_q     1.0000    2.07e-15   4.82e+14    0.000    1.000    1.000
milk_q          1.0000    1.01e-16   9.92e+15    0.000    1.000    1.000
onion_q         1.0000    1.67e-15    6e+14     0.000    1.000    1.000
potato_q        1.0000    1.38e-15   7.27e+14    0.000    1.000    1.000
=====
Omnibus:             584.821   Durbin-Watson:           0.357
Prob(Omnibus):        0.000   Jarque-Bera (JB):        12354.080
Skew:                 -0.281   Prob(JB):                 0.00
Kurtosis:             12.735   Cond. No.                 164.
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
# Calculate predictions
```

```
predictions = results.predict(sm.add_constant(test[X.columns]))
```

```
# Calculate R-squared
r_squared = np.corrcoef(predictions, test["tot_con"])[0, 1]**2
print("R-squared:", r_squared)

R-squared: 1.0

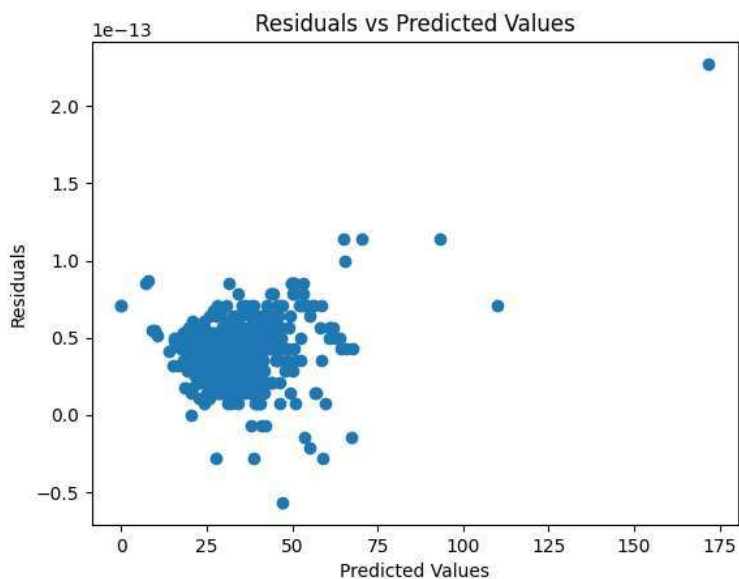
# Define MAPE function
def MAPE(actual, predicted):
    non_zero_actual = actual[actual != 0]
    non_zero_predicted = predicted[actual != 0]
    return np.mean(np.abs((non_zero_actual - non_zero_predicted) / non_zero_actual)) * 100

# Calculate MAPE
mape_value = MAPE(test["tot_con"], predictions)
print("MAPE:", mape_value)

MAPE: 1.3124859102165508e-13

# Residual analysis
residuals = predictions - test["tot_con"]

# Plot residuals against predicted values
plt.scatter(predictions, residuals)
plt.xlabel("Predicted Values")
plt.ylabel("Residuals")
plt.title("Residuals vs Predicted Values")
plt.show()
```



```
# Homoscedasticity Test - Breusch-Pagan test
bp_test = sm.stats.diagnostic.het_breuschpagan(residuals, sm.add_constant(predictions))
print("Breusch-Pagan p-value:", bp_test[1])
```

Breusch-Pagan p-value: 2.55668563776929e-41

```
import pandas as pd
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

# Check for multicollinearity using VIF
vif = pd.DataFrame()
vif["Variable"] = X.columns
vif["VIF"] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
print(vif)

# Address heteroscedasticity using robust standard errors
model = sm.OLS(y, X)
```

```
results = model.fit(cov_type='HC3') # HC3 provides heteroscedasticity-consistent standard errors
print(results.summary())
```

Variable	VIF
0 wheattotal_q	45.101823
1 cerealtot_q	63.080724
2 moong_q	3.402170
3 pulsestot_q	8.091110
4 milk_q	3.617673
5 onion_q	7.615561
6 potato_q	7.898856

=====						
Dep. Variable:	tot_con	R-squared (uncentered):	1.000			
Model:	OLS	Adj. R-squared (uncentered):	1.000			
Method:	Least Squares	F-statistic:	4.041e+33			
Date:	Wed, 07 Jun 2023	Prob (F-statistic):	0.00			
Time:	14:39:45	Log-Likelihood:	95246.			
No. Observations:	3118	AIC:	-1.905e+05			
Df Residuals:	3111	BIC:	-1.904e+05			
Df Model:	7					
Covariance Type:	HC3					
=====						
	coef	std err	z	P> z	[0.025	0.975]

wheattotal_q	1.0000	2.91e-16	3.43e+15	0.000	1.000	1.000
cerealtot_q	1.0000	2.81e-16	3.56e+15	0.000	1.000	1.000
moong_q	1.0000	2.39e-15	4.18e+14	0.000	1.000	1.000
pulsestot_q	1.0000	7.1e-16	1.41e+15	0.000	1.000	1.000
milk_q	1.0000	9.22e-17	1.08e+16	0.000	1.000	1.000
onion_q	1.0000	7.06e-16	1.42e+15	0.000	1.000	1.000
potato_q	1.0000	4.78e-16	2.09e+15	0.000	1.000	1.000
=====						
Omnibus:	1241.580	Durbin-Watson:	0.962			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	12029.288			
Skew:	1.617	Prob(JB):	0.00			
Kurtosis:	12.063	Cond. No.	164.			
=====						

Notes:

- [1] R² is computed without centering (uncentered) since the model does not contain a constant.
 [2] Standard Errors are heteroscedasticity robust (HC3)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
```

```
from google.colab import files
uploaded = files.upload()
```

IPL salary 2...8_2022.xlsx
 • **IPL salary 2008_2022.xlsx**(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 235367 bytes, last modified: 6/7/2023 - 100% done
 Saving IPL salary 2008_2022.xlsx to IPL salary 2008_2022.xlsx

```
ipl_salary = pd.read_excel('IPL salary 2008_2022.xlsx')
```

```
ipl_salary = ipl_salary[['Runs', 'Wkts', 'Final Price']].dropna()
```

```
X = ipl_salary[['Runs', 'Wkts']]
y = ipl_salary['Final Price']
X = sm.add_constant(X) # Add constant term
model = sm.OLS(y, X)
result = model.fit()
print(result.summary())
```

=====			
Dep. Variable:	Final Price	R-squared:	0.248
Model:	OLS	Adj. R-squared:	0.247
Method:	Least Squares	F-statistic:	182.6

```

Date:           Wed, 07 Jun 2023   Prob (F-statistic):       2.98e-69
Time:           14:45:36          Log-Likelihood:         -20491.
No. Observations: 1108           AIC:                     4.099e+04
Df Residuals:    1105           BIC:                     4.100e+04
Df Model:        2
Covariance Type: nonrobust

```

```

=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
const      9.724e+06    1.21e+06     8.053     0.000     7.35e+06    1.21e+07
Runs      9.656e+04    5084.685    18.991     0.000     8.66e+04    1.07e+05
Wkts      8.28e+05    1.33e+05     6.221     0.000     5.67e+05    1.09e+06
=====
Omnibus:                 263.060    Durbin-Watson:           0.944
Prob(Omnibus):            0.000    Jarque-Bera (JB):        592.892
Skew:                     1.300    Prob(JB):                1.80e-129
Kurtosis:                 5.466    Cond. No.                309.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

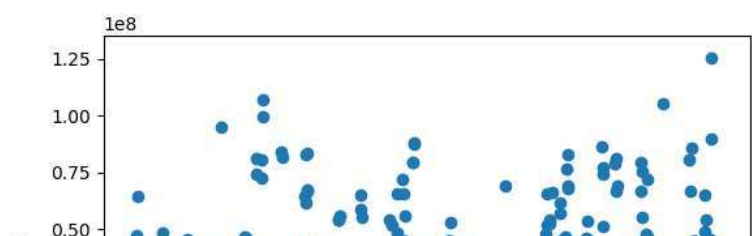
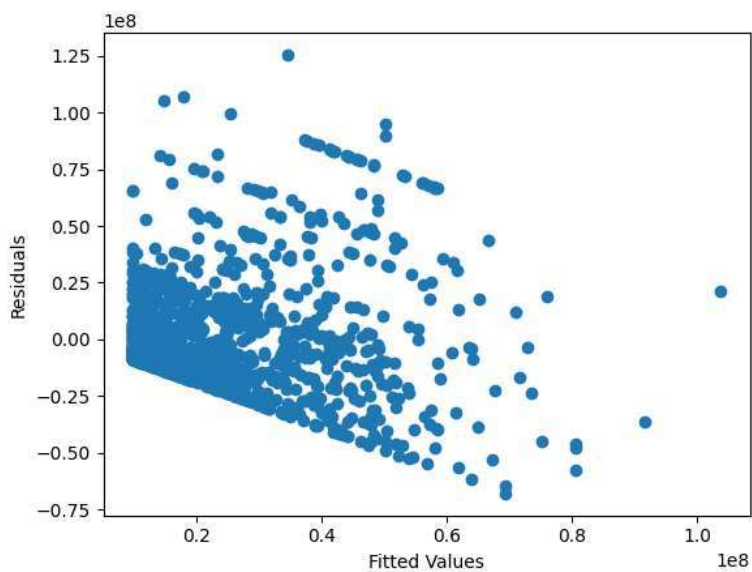
```

# Residual plot for linearity
plt.scatter(result.fittedvalues, result.resid)
plt.xlabel("Fitted Values")
plt.ylabel("Residuals")
plt.show()

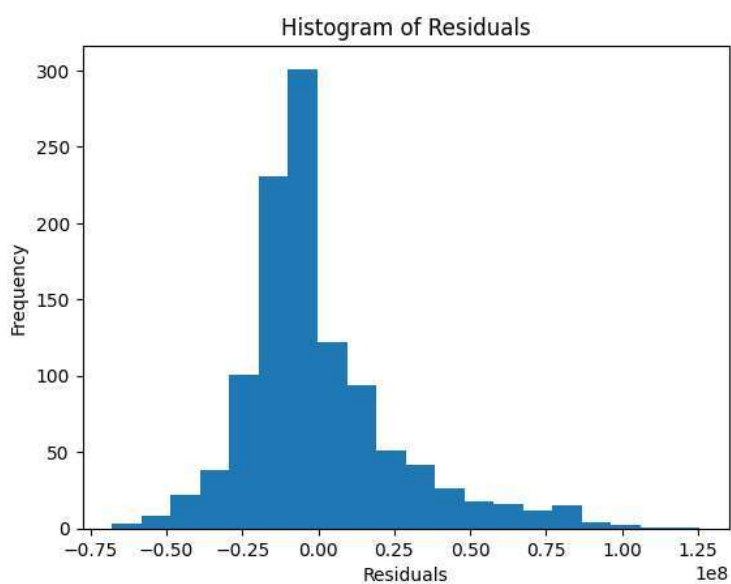
# Residual plot for independence
plt.scatter(range(len(result.resid)), result.resid)
plt.xlabel("Index")
plt.ylabel("Residuals")
plt.show()

# Residual plot for homoscedasticity
plt.scatter(result.fittedvalues, np.sqrt(abs(result.resid)))
plt.xlabel("Fitted Values")
plt.ylabel("Square Root of Residuals")
plt.show()

```



```
plt.hist(result.resid, bins=20)
plt.xlabel("Residuals")
plt.ylabel("Frequency")
plt.title("Histogram of Residuals")
plt.show()
```



```
sm.qqplot(result.resid, line='s')
plt.xlabel("Theoretical Quantiles")
plt.ylabel("Sample Quantiles")
plt.show()
```



